



AIRSHIELD

PRIVATE AI CONTROL PLANE

IMPLEMENTATION • OPERATIONS • ACCEPTANCE

Requirements and Runbook

Implemented scope, exact run steps, verification and production gates

**BUILD IT. RUN IT. VERIFY IT. RELEASE
IT SAFELY.**

CAPTURE

PROTECT

AUTHORIZE

PROVE

Contents

Implemented scope, exact run steps, verification and production gates

1. Purpose, audience and status	3
2. Product scope and requirements	3
3. CareShield Assistant acceptance criteria	4
4. Repository map	5
5. Prerequisites	5
6. Fastest run: UI and development protection	6
7. Full local stack: recommended voice demonstration	6
8. Demonstrate CareShield safely	7
8A. Demonstrate EgressSeal, Destination Switch, ContextFence and SafeAction	8
9. Demonstrate Live Shield	8
10. Manual control-plane development	8
11. Validation procedure	9
12. Troubleshooting	10
13. Production deployment sequence	10
14. Release acceptance checklist	11
15. Source documents for deeper review	11

1. Purpose, audience and status

This runbook is for product owners, developers, reviewers, demo operators, security teams and release engineers. It covers the responsive Next.js experience, Python control plane, self-hosted voice edge, authenticated gateway, local stack, CareShield Assistant demonstration and production reference assets.

The repository is a serious control-plane prototype, not a compliance certificate, medical device, clinical system, emergency service or real appointment platform. It does not by itself establish GDPR, HIPAA, PCI DSS, DORA, NIS2, EHDS, EU AI Act, SOC 2 or ISO compliance. Production use requires customer-specific legal, privacy, security, model, penetration, infrastructure and applicable independent assessment.

2. Product scope and requirements

ID	Requirement	Implemented evidence
FR-01	Protect English text before approved AI egress	Next.js server route and Python control-plane <code>/v1/protect</code> path
FR-02	Capture voice without paid transcription APIs	Self-hosted <code>faster-whisper</code> voice edge over WebSocket
FR-03	Show live raw and protected transcripts side by side	Live Shield trust-boundary and outbound panes
FR-04	Apply sector-aware policy	Healthcare, Finance, Insurance, Contact Center and Internal Copilot policies
FR-05	Keep raw identifiers inside the controlled boundary	Browser/edge/control-plane boundary; protected text is the only intended downstream payload
FR-06	Tokenize reversibly when policy permits	Encrypted token mappings in the Python vault
FR-07	Produce evidence for decisions	Signed receipt metadata and tenant hash-chain design
FR-08	Support controlled reidentification	Request, separate approval, expiry and one-time result retrieval APIs
FR-09	Map speakers without treating diarization as identity	Host authentication plus signed isolated-channel binding; unknown stays unknown
FR-10	Integrate with common application stacks	OpenAPI/AsyncAPI plus Java, .NET, Python, Node.js and Go examples
FR-11	Run in Azure-private and cloud-neutral environments	AKS/Key Vault/PostgreSQL and Kubernetes/OpenBao deployment references
FR-12	Fail closed in production	Startup/runtime guards reject development auth, local keys, insecure URLs and missing dependencies
FR-13	Bind outbound authorization to content and destination	EgressSeal™ signs the protected digest, policy, destination, risk, receipt, expiry and action scope
FR-14	Recalculate when the destination changes	Destination Switch invalidates the prior seal and requests a new destination-specific decision
FR-15	Detect cumulative post-masking risk	ContextFence™ scores explainable token-linkage, entity-combination, semantic and quasi-identifier factors
FR-16	Demonstrate token-safe actions	SafeAction™ verifies the seal and runs an allowlisted synthetic connector without exposing raw values to the AI

Non-functional requirements

- Desktop, tablet and mobile layouts must remain usable from a minimum 320-pixel viewport.
- The product must not silently call browser or paid cloud speech-recognition services.
- Production raw content must not enter application logs, receipts, metrics or analytics.

- Tenant, policy, destination, identity and cryptographic context must be explicit at the decision point.
- Request size, connection count, rate, inference concurrency and session duration must be bounded.
- Production components must authenticate workloads and use TLS; browser voice must use authenticated `wss://`.
- Detector, ASR and diarization releases must be pinned and evaluated against representative approved data.

3. CareShield Assistant acceptance criteria

The approved CareShield Assistant is an initially open, bottom-right widget added without removing or replacing any existing AirShield view.

ID	Acceptance criterion	Expected behavior
CS-01	Open, minimize, restore and reset	Header controls preserve the underlying AirShield application and launcher restores the panel
CS-02	Text command path	User enters symptoms; only the local user bubble shows raw text
CS-03	Protected RIA preview	The bot creates an AirShield session, protects the text and shows tokenized/masked outbound content, entity count, decision and receipt identifier
CS-04	Voice-first path	Microphone audio is streamed to the configured self-hosted voice WebSocket; transcript pairs show local raw and protected text
CS-05	Guided flow	Progress advances through Symptoms, Doctor and Book
CS-06	Doctor selection	A synthetic virtual general-physician slot is displayed after intake
CS-07	Reservation demonstration	Selecting the slot creates a clearly labelled demo reservation only
CS-08	Emergency safety	The widget states it is not emergency care and provides urgent local-emergency guidance when warning signs are selected or detected
CS-09	Clinical limitation	The widget states it does not diagnose, create a clinical record or make a real appointment
CS-10	Responsive behavior	Floating desktop panel, tablet-safe panel and foreground mobile panel remain operable with the existing navigation

The CareShield route is a demonstration of protected integration. The repository does not contain a real EHR, clinician directory, appointment inventory, RIA connector or booking transaction.

EgressSeal innovation acceptance criteria

ID	Acceptance criterion	Expected behavior
ES-01	Destination-bound release proof	Seal payload binds protected-content SHA-256, upstream receipt, policy, destination, ContextFence result, expiry and action allowlist
ES-02	Cryptographic verification	Development API issues and verifies an Ed25519 signature and rejects changed protected content or expired/untrusted seals
ES-03	Destination Switch	Selecting a new destination prevents the prior destination-bound seal from authorizing the new route and recalculates exposure/risk
ES-04	ContextFence	UI explains cumulative token linkage, entity combinations, semantic context, quasi-identifiers, conversation length and relationship factors
ES-05	Fail-closed destination	Public general AI is blocked even after field-level protection

ID	Acceptance criterion	Expected behavior
ES-06	SafeAction	Broker verifies seal, content digest, destination and action allowlist before returning a signed synthetic action receipt
ES-07	No model-side raw resolution	SafeAction uses protected token references; the synthetic connector never returns raw values to the model
ES-08	Honest development boundary	UI states that the process-local key and connector are hackathon demonstrations, not a production trust anchor or transaction

EgressSeal™, ContextFence™ and SafeAction™ are product-concept names, not claims of registered trademark status. Production requires KMS/HSM signing, trusted upstream-receipt verification, durable key rotation, separately authorized connectors and independent security review.

4. Repository map

Path	Purpose
<code>app/, components/, lib/</code>	Next.js App Router UI, CareShield widget and server-only proxy routes
<code>control-plane/</code>	FastAPI policy, detection, encrypted vault, evidence, retention and reidentification
<code>gateway-service/</code>	OIDC-authenticated external WebSocket boundary and private edge proxy
<code>edge-service/</code>	Self-hosted English ASR/optional diarization adapter
<code>api/</code>	OpenAPI and AsyncAPI contracts
<code>examples/</code>	Java, .NET, Python, Node.js and Go client examples
<code>deploy/kubernetes/</code>	Hardened base and Azure/OpenBao overlays
<code>deploy/azure/</code>	Private Azure reference built around AKS, Key Vault and PostgreSQL
<code>docs/</code>	Architecture, threat model, model assurance, operational evidence and release guidance
<code>.env.local</code>	Tracked non-secret local browser voice configuration

5. Prerequisites

UI and protected-text demonstration

- Node.js 22 or later
- npm 10 or later
- A current Chromium, Firefox or Safari-class browser

Complete local voice stack

- Docker Engine with Compose v2
- At least 8 GB of free memory for a CPU demonstration; more is recommended
- Network access on first development model download
- Browser microphone permission
- `localhost` or HTTPS, because browsers restrict microphone access in insecure remote contexts

Manual backend development

- Python 3.12 or later; the production image targets Python 3.12
- PostgreSQL 16 or 17
- `ffmpeg` and `libsndfile` for voice processing

- Optional NVIDIA-supported runtime when using an evaluated GPU configuration

6. Fastest run: UI and development protection

From the repository root:

```
npm ci
npm run dev
```

Open:

```
http://localhost:4174
```

Expected result:

- 1 The responsive AirShield Overview opens with all existing product views available.
- 2 The CareShield Assistant is open at the bottom-right; minimize it to use the underlying views.
- 3 Typed protection uses the explicitly enabled local development detector when no control plane is configured.
- 4 `/api/health` reports `development-prototype` rather than pretending the production control plane is ready.
- 5 Microphone transcription does not work until the self-hosted edge is running on port `8001`.

The root `.env.local` contains non-secret hackathon defaults only:

```
NEXT_PUBLIC_EDGE_WS_URL=ws://localhost:8001/ws/voice
AIRSHIELD_RUNTIME_MODE=development
AIRSHIELD_ALLOW_DEVELOPMENT_RUNTIME=true
AIRSHIELD_ALLOW_INSECURE_LOCAL_EDGE=true
```

`NEXT_PUBLIC_*` values are embedded when Next.js starts or builds. Restart the development server, or rebuild the image, after changing the voice URL.

7. Full local stack: recommended voice demonstration

Run from the repository root:

```
docker compose up --build
```

The development stack starts:

Service	Address	Purpose
Web	http://localhost:4174	Responsive UI and protected server routes
Control plane	http://localhost:8080	Policy, vault and signed evidence path
Voice edge	ws://localhost:8001/ws/voice	Self-hosted English voice capture
PostgreSQL	Compose private network	Development persistence

The Compose configuration intentionally contains visible development credentials, local key material and development authorization. It is unsafe for production.

Optional local model destination:

```
docker compose --profile local-llm up --build
```

Stop services while retaining volumes:

```
docker compose down
```

Stop services and delete development data/model volumes:

```
docker compose down -v
```

8. Demonstrate CareShield safely

Text flow

- 1 Open <http://localhost:4174> and leave CareShield open.
- 2 Select a suggested command or enter a synthetic sentence such as: [I am Jordan Lee, my phone is 555-010-8832, and I have had a headache since yesterday.](#)
- 3 Select send.
- 4 Confirm the raw sentence appears only in the right-aligned [You · local only](#) bubble.
- 5 Confirm the [Protected for RIA demo](#) card shows protected/tokenized content, a policy decision, entity count and receipt reference.
- 6 Confirm a virtual general-physician slot appears.
- 7 Select [Reserve demo](#) and verify that the confirmation says no real appointment or clinical record was created.
- 8 Select reset before the next demonstration.

Use synthetic data only. Do not enter real health, payment or identity information into a hackathon environment.

Voice flow

- 1 Start the full Compose stack and wait for the edge health check to pass.
- 2 Open CareShield and select the microphone button.
- 3 Allow browser microphone access.
- 4 Speak one short English synthetic symptom statement.
- 5 Select the red microphone/stop control.
- 6 Confirm local raw transcript and protected outbound transcript are presented separately.
- 7 Confirm the final decision and receipt arrive before treating the turn as egress-eligible.

If the browser page is not on [localhost](#), serve it over HTTPS and configure an authenticated reachable [wss://](#) gateway. An HTTPS page cannot use [ws://](#), and a remote browser cannot reach a service through the server's [localhost](#) address.

Emergency branch

Select [Emergency warning signs](#) or enter obvious emergency language. The widget must advise contacting local emergency services immediately. It must not collect additional details, diagnose, promise treatment or imply clinician monitoring.

8A. Demonstrate EgressSeal, Destination Switch, ContextFence and SafeAction

- 1 Open **EgressSeal™** from the desktop sidebar or mobile **Seal** tab.
- 2 Select an industry and retain the supplied synthetic text.
- 3 Select **Organization private AI**, then select **Protect & request seal**.
- 4 Confirm that only the protected payload appears in the outbound preview.
- 5 Review the ContextFence score and its explainable cumulative-risk factors.
- 6 Confirm that EgressSeal binds content digest, upstream receipt, policy, destination, risk, expiry and one allowlisted action.
- 7 Select **Verify EgressSeal**; SafeAction must remain locked until signature, digest and expiry verification succeeds.
- 8 Select **Run SafeAction** and confirm the broker reports token-only input, connector-only resolution, no raw value returned to the model and a signed synthetic action receipt.
- 9 Switch to **Public general AI** and request another seal; the destination must be blocked.
- 10 Switch destinations after an issued seal and confirm the prior destination-bound authorization cannot authorize the newly selected route.

The development route uses a process-local receipt registry and Ed25519 key, so restart invalidates existing receipts and seals. It accepts only receipts created by `/api/protect` whose protected digest, decision, policy and destination still match. It is a real cryptographic protocol demonstration but not a production authority. Production must use a pinned external KMS/HSM trust anchor, verify the upstream control-plane receipt itself, enforce workload identity and run the action broker as separately secured infrastructure.

9. Demonstrate Live Shield

- 1 Open **Live Shield** from the sidebar or mobile navigation.
- 2 Select the Healthcare, Finance, Insurance, Contact Center or Internal Copilot scenario.
- 3 Select **Start live capture**; microphone permission is requested automatically.
- 4 Speak synthetic English identifiers.
- 5 Confirm the left pane is labelled as raw/local and the right pane is protected/outbound.
- 6 Select **Stop & protect** to force processing of the final short audio chunk.
- 7 Verify the final full transcript has an **allow**, **block** or **review** decision and a receipt.
- 8 Select **Reset** to release the microphone, close the WebSocket and clear both panes.

Interim transcript pairs are provisional and must not be sent downstream. Only a signed final **allow** after complete-turn rechecking is intended to be egress-eligible.

10. Manual control-plane development

Create an isolated Python environment:

```
cd control-plane
python3.12 -m venv .venv
source .venv/bin/activate
pip install -e '[dev]'
python -m app.seed_dev_keys
```

Copy the generated development values to a private uncommitted `control-plane/.env`, configure a PostgreSQL development URL, then run:

```
alembic upgrade head
uvicorn app.main:app --host 0.0.0.0 --port 8080 --no-proxy-headers
```

Development-only settings are:

```
ENVIRONMENT=development
AUTH_MODE=dev
KEY_PROVIDER=local
CONTROL_PLANE_URL=http://127.0.0.1:8080
CONTROL_PLANE_DEV_TOKEN=development-only
```

Never carry those values into production. Production startup is designed to reject development auth, local key providers, raw-response debug behavior, insecure upstream URLs and other unsafe combinations.

Verify readiness:

```
curl http://127.0.0.1:8080/v1/health/ready
```

Run the example clients from the repository root:

```
python examples/python/client.py
node examples/node/client.mjs
```

11. Validation procedure

Web checks

```
npm ci
npm audit --audit-level=high
npm run typecheck
npm run build
```

Python control-plane checks

```
cd control-plane
python3.12 -m venv .env
source .env/bin/activate
pip install -e '[dev]'
ruff format --check app tests migrations evaluation scripts
ruff check app tests migrations evaluation scripts
mypy app scripts
pytest -q
python evaluation/evaluate.py evaluation/fixtures/regex-gate.jsonl --require-perfect
alembic upgrade head --sql
```

Voice edge and gateway checks

```
(cd edge-service && PYTHONPATH=. ../control-plane/.env/bin/pytest -q tests)
(cd gateway-service && PYTHONPATH=. ../control-plane/.env/bin/pytest -q tests)
```

Manual responsive checks

Test widths of approximately 1440, 1024, 768, 430 and 320 pixels. At each width verify navigation, Live Shield transcript panes, controls, modals, tables, CareShield open/minimized states, CareShield scrolling, text entry and reservation demonstration. Verify keyboard focus and reduced-motion behavior.

The included synthetic detector evaluator is a regression smoke test. It deliberately does not satisfy the production model gate. Representative, legally sourced and approved Nordic/EU and US English evaluation data is mandatory before production promotion.

12. Troubleshooting

Symptom	Likely cause	Action
Voice edge unavailable	Edge is not running or URL was embedded incorrectly	Start Compose; verify port <code>8001</code> ; restart/rebuild Next.js after changing <code>.env.local</code>
Microphone denied	Browser permission or insecure context	Allow microphone; use <code>localhost</code> or HTTPS
HTTPS page cannot connect	Browser blocks <code>ws://</code> mixed content	Use authenticated trusted <code>wss://</code>
Remote demo cannot reach edge	Browser resolves <code>localhost</code> on its own device	Publish a reachable gateway URL and exact allowed origin
Protection returns 503	Production mode has no configured control plane, or upstream is unavailable	Configure authenticated <code>CONTROL_PLANE_URL</code> ; check readiness and TLS
CareShield text stays pending	<code>/api/sessions</code> or <code>/api/protect</code> failed	Inspect server response; verify development runtime or control plane
Voice has no final transcript	Session stopped before finalization or model startup is incomplete	Wait for model readiness, speak a short full sentence, then stop once
UI ignores changed voice URL	<code>NEXT_PUBLIC_*</code> was already embedded	Restart development server or rebuild web image
Production startup fails	Fail-closed guard found an unsafe/missing setting	Correct identity, key, TLS, CORS, model, database and audit configuration; do not disable the guard

13. Production deployment sequence

- 1 Build, scan, sign, attest and digest-pin web, gateway, edge and control-plane images.
- 2 Build the web image with the exact public authenticated `wss://.../ws/voice` gateway URL.
- 3 Provision web/gateway/edge/control-plane certificates and private CA bundles.
- 4 Provision PostgreSQL with `ssl=verify-full`, tested backup/restore and tenant-safe migrations.
- 5 Provision separated control-plane and gateway secret stores with distinct managed/workload identities.
- 6 Configure Azure Key Vault or OpenBao Transit; never use local wrapping in production.
- 7 Preload evaluated ASR/diarization artifacts on read-only storage and verify a promoted signed manifest before model load.
- 8 Replace all `REPLACE_*` identity, tenant, image, DNS, vault and certificate placeholders.
- 9 Render and validate Azure or OpenBao overlays; run Azure what-if where applicable.
- 10 Apply Alembic through an approved one-shot job, then deploy.
- 11 Test direct-edge denial, origin/tenant isolation, token expiry, secret and certificate rotation, dependency outages, model tampering, network policies, restore, retention/deletion and immutable evidence export.
- 12 Complete customer-specific legal, privacy, security, model, penetration and applicable sector assessments.

Reference validation commands:

```
kustomize build deploy/kubernetes/overlays/azure | kubeconform -strict -ignore-missing-schemas -
kustomize build deploy/kubernetes/overlays/openbao | kubeconform -strict -ignore-missing-schemas -
az bicep build --file deploy/azure/main.bicep
```

14. Release acceptance checklist

- ☐ Existing AirShield screens and features remain available.
- ☐ CareShield open, minimize, restore and reset behavior works on desktop, tablet and mobile.
- ☐ Typed raw content appears only in the local bubble; protected content is used for the downstream demonstration.
- ☐ Self-hosted voice path works with no cloud transcription fallback.
- ☐ Emergency and no-diagnosis/no-real-booking language is visible.
- ☐ Web typecheck, production build and dependency audit pass.
- ☐ Python, edge and gateway tests pass in a clean supported environment.
- ☐ No real personal, health or payment data is used in the demo.
- ☐ Development credentials and local keys are absent from production configuration.
- ☐ Production identity, TLS, key custody, detector/model, retention and audit export controls are validated.
- ☐ Independent security/privacy review and applicable regulatory assessment are complete.
- ☐ Deployment rollback and incident procedures have named owners.

15. Source documents for deeper review

- docs/AIRSHIELD_HACKATHON_PITCH.md
- docs/AIRSHIELD_MANAGER_DEMO_PLAYBOOK.md
- docs/AIRSHIELD_TECHNICAL_AUTHORITY_DOSSIER.md
- docs/AIRSHIELD_CEO_TECHNICAL_BRIEFING.md
- docs/PRODUCTION_ARCHITECTURE.md
- docs/THREAT_MODEL.md
- docs/MODEL_ASSURANCE.md
- docs/OPERATIONS_EVIDENCE.md
- docs/DPIA_CONTROL_MAPPING.md
- docs/SECURITY_REVIEW_CHECKLIST.md
- deploy/kubernetes/README.md
- deploy/azure/README.md
- api/openapi.yaml and api/asynapi.yaml

Release rule: no allow without an evaluated detector, authenticated tenant/destination context and verifiable final receipt. Missing identity, keys, model, policy, database or audit dependencies must fail closed rather than silently bypass AirShield.